



Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**



Grado en Ingeniería Informática

Trabajo Fin de Grado

**Booster: Una nueva plataforma de
videos de código abierto**

Autor: Samuel Caraballo Chichiraldi
Tutor: Fernando Pérez Costoya

Madrid, Junio 2026

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado
Grado en Ingeniería Informática

Título: Booster: Una nueva plataforma de videos de código abierto
Junio 2026

Autor: Samuel Caraballo Chichiraldi
Tutor: Fernando Pérez Costoya
Departamento de Arquitectura y Tecnología de Sistemas Informáticos
Escuela Técnica Superior de Ingenieros Informáticos
Universidad Politécnica de Madrid

Resumen

El trabajo aborda el diseño y la implementación de una red social de vídeos en formato web llamada Booster. Se detallará el funcionamiento de la infraestructura de la plataforma así como las tecnologías utilizadas. Además, se hará una evaluación de las distintas opciones para procesar los vídeos: Bunny.net, Mux y AWS. También se documentará el código, las decisiones de diseño y se realizará una guía de despliegue.

El objetivo del proyecto es implementar un prototipo (Producto Mínimo Viable) y detallar cómo funcionan todas las partes de la plataforma de vídeos:

- autenticación
- arquitectura del procesamiento y transcodificación de vídeos
- despliegue,
- diseños de las interfaces de usuario
- optimizaciones
- riesgos de seguridad
- creación de comentarios
- notificaciones
- diseño del algoritmo de recomendación
- creación de comunidades
- carga de vídeos y edición de sus metadatos
- recuento de visitas
- calificaciones a los vídeos
- filtrado de los vídeos por categorías
- recuento de los seguidores de un usuario
- explicación del sistema de Boost y cómo funciona la moneda virtual (XP)
- diseño de la base de datos
- conexión entre front-end y back-end y explicación de las tecnologías usadas:

-
- Next.js
 - tRPC
 - NeonDB
 - DrizzleORM
 - SupaBase
 - TailwindCSS
 - ReactJS

Abstract

Abstract of the Final Degree Project. Maximum length: 2 pages.

Tabla de contenidos

1. Introducción	1
1.1. Motivación y necesidades del proyecto	1
1.2. Objetivos	3
1.3. Estructura de la memoria	3
2. Estado del Arte	7
2.1. Procesamiento, Transcodificación y Entrega de Video	7
2.1.1. FFmpeg	7
2.1.2. MPEG-DASH	9
2.1.3. HLS	10
2.1.4. Object Storage	11
2.1.5. CDN - Content Delivery Network	12
2.1.6. Colas de Procesamiento	14
2.2. Análisis de Proveedores	15
2.2.1. Amazon Web Services y solución propia	15
2.2.2. Mux.com	17
2.2.3. Bunny.net	19
2.2.4. Tabla comparativa de proveedores	21
2.3. Modelos de Arquitectura	21
2.3.1. Modelo Centralizado	22
2.3.2. Modelo Federado	24
2.3.3. Modelo Peer-to-Peer (P2P)	26
3. Tecnologías Empleadas	29
3.1. Backend	29
3.1.1. TypeScript como lenguaje de programación	29
3.1.2. Base de Datos	31
3.1.3. ORMs	33
3.1.4. Comunicación Cliente Servidor - tRPC	35
3.1.5. Next.js - backend	37
3.1.6. Autenticación - ClerkAuth	39
3.2. Frontend	40
3.2.1. React.Js	40
3.2.2. Next.Js - frontend	41
3.2.3. TailwindCSS y Shadcn	41

TABLA DE CONTENIDOS

4. Innovaciones	43
4.1. Problemas detectados y la solución de Booster	43
4.1.1. Problemas con el algoritmo de recomendación	43
4.1.2. Problemas en encontrar comunidades	43
4.2. Mejoras e Innovaciones	43
4.2.1. Mejoras en la interfaz de usuario	43
4.2.2. Mejoras en los métodos de moderación	43
4.2.3. Mejoras en los comentarios	43
4.2.4. Mejoras en los canales de usuario	43
4.2.5. Mejoras en las comunidades	43
4.2.6. Incorporación de gamificación	43
5. Análisis	45
5.1. Requisitos Funcionales	45
5.2. Requisitos No Funcionales	45
5.3. Actores	45
5.4. Casos de uso	45
5.5. Diagramas de secuencia	45
5.6. Diagrama Entidad Relación	45
6. Diseño y Arquitectura	47
6.1. Patrones y mejores prácticas	47
6.2. Diseño de la Autenticación	47
6.3. Diseño de la Base de Datos	47
6.4. Arquitectura de procesamiento propia	47
6.5. Arquitectura de procesamiento con un proveedor	47
6.6. Diseño de la interfaz de usuario	47
6.6.1. Comentarios	47
6.6.2. Interfaz de Video	47
6.6.3. Secciones de Video	47
6.7. Diseño del Algoritmo de Recomendación	47
6.8. Diseño de la búsqueda semántica de videos	47
7. Desarrollo	49
7.1. Implementación del procesamiento de videos propio	49
7.2. Implementación del procesamiento de videos con proveedor	49
7.3. Implementación de los comentarios	49
7.4. Patrones y mejores prácticas en código	49
7.5. Acceder y modificar datos de usuario	49
7.6. Acceder y modificar datos de video	49
7.7. Medidas de seguridad	49
7.7.1. Protección acceso a rutas	49
7.7.2. Protección acceso y modificación de videos	49
7.7.3. Limitaciones de uso (Rate Limits)	49
8. Pruebas, Validación y Riesgos de Seguridad Detectados	51
8.1. Pruebas de funcionamiento	51
8.2. FeedBack de Usuarios	51

8.3. Vulnerabilidades detectadas y medidas tomadas	51
9. Conclusiones	53
10. Análisis de Impacto	55
Bibliografía	57
Anexos	61
A. Primer anexo	61

Capítulo 1

Introducción

1.1. Motivación y necesidades del proyecto

Los servicios de video bajo demanda se encuentra en una fase de crecimiento continuo y altísima intensidad de uso, con YouTube, TikTok, Netflix y los Reels de Meta siendo las principales plataformas dominantes del sector. Esto es confirmado con los datos de Google [1] el cual reportó que los ingresos de YouTube por publicidad y suscripciones superaron los 60 mil millones de dólares en 2025, teniendo un alcance publicitario mensual de 2.53 mil millones de usuarios a inicios de 2025, como fue demostrado por DataReportal.

Asimismo, en abril de ese mismo año YouTube informó que la plataforma acumulaba más de 20 mil millones de videos subidos, que recibía más de 20 millones de videos nuevos al día, y que en 2024 promedió más de 100 millones de comentarios diarios y más de 3.5 mil millones de “likes” por día. Esto demuestra no sólo la escala, sino también la intensidad de uso de las plataformas de este estilo.

De forma similar, TikTok [2] confirmó que en 2025 ya superaba los 200 millones de usuarios mensuales en Europa. Por otro lado, estimaciones de eMarketer exponían el tiempo promedio que destinaba un usuario en la plataforma por día, con cifras que rondaban los 52 minutos diarios por usuario.

Netflix, por su parte, informó [3] que recibió más de 325 millones de membresías pagadas y 96 mil millones de horas vistas en el segundo semestre de 2025. Eso equivale a aproximadamente 522 millones de horas reproducidas por día durante ese semestre, lo cual es un signo claro de la persistencia de consumo y la alta inversión temporal que dedican los usuarios.

En Meta, la situación es similar. La empresa [4] reportó 3.58 mil millones de personas activas por día en su familia de aplicaciones en diciembre de 2025 y comunicó, adicionalmente, que en Estados Unidos el tiempo de visualización de video en Facebook creció más de 20 por ciento interanual, lo que refleja el consumo masivo de videos en línea y el dominio del video de formato corto basado en algoritmos que priorizan el contenido dinámico.

Capítulo 1. Introducción

Es importante destacar, el impacto medioambiental que tiene este consumo masivo de contenido en línea. Un estudio realizado por La Agencia Internacional de la Energía [5] estimó que los centros de datos consumieron alrededor de 415 TWh en 2024, equivalentes a cerca de 47.4 GW de demanda media lo cual se aproxima a 1.5% del consumo mundial de electricidad. Aunque esa cifra no se puede atribuir exclusivamente al contenido de video en línea, un estudio de Global Internet Phenomena de AppLogic confirmó que el streaming de video continúa siendo la principal fuente de tráfico en internet, con 38% del total global y que las aplicaciones de redes sociales de videos en línea y entretenimiento dominan el volumen de red por internet.

Estos datos, demuestran el estado actual de las plataformas de streaming de video bajo demanda, el alcance masivo que tienen y su elevadísima frecuencia de uso. No son solo aplicaciones de entretenimiento sino, servicios digitales con infraestructuras extremadamente complejas capaces de concentrar a miles de millones de personas, acumular millones de horas de video y moldear los hábitos de entretenimiento de la sociedad.

No obstante, cuanto más dominantes se han vuelto estas plataformas, más visibles se han hecho las quejas y los descontentos de sus usuarios.

En cuanto al sesgo, las cámaras de eco y la sensación de encierro algorítmico, los datos son significativos. Ofcom [6] encontró que alrededor de un tercio de los usuarios manifiesta incomodidad explícita con los algoritmos de recomendación y que, a su vez, los algoritmos no suelen favorecer las opiniones con las que discrepa un usuario.

Con respecto al contenido producido por IA, contenido inapropiado y desinformación, la escala del malestar es incluso más apreciable. Ofcom [7] reportó que 41% de los adultos usuarios de internet en el Reino Unido encontró desinformación en las cuatro semanas previas a la encuesta, 22% vio imágenes o videos falsos o engañosos, y 25% dijo haber quedado realmente molesto u ofendido por el daño más reciente encontrado online. Además, 47% nunca había visto herramientas, etiquetas o íconos que le ayudaran a entender si un contenido había sido creado o editado con IA, aunque 85% consideró importante que las plataformas lo informaran.

Asimismo, un estudio académico publicado en European Journal of Operational Research en 2025 [8], demuestra un malestar significativo de los usuarios frente a la sobrecarga publicitaria en redes sociales de video. Tomando datos globales sobre uso de bloqueadores de anuncios, señala que las razones más frecuentes para bloquear anuncios son la cantidad excesiva de anuncios (62.1%) y su carácter intrusivo (54.2%), lo que constituye una señal clara de fatiga publicitaria en entornos digitales de streaming de video. A ello se suma que el estudio global Media Reactions 2025 de Kantar muestra que la percepción de una plataforma mejora cuando sus anuncios son vistos como menos intrusivos, como ocurrió con Snapchat. Esto confirma que la intrusividad publicitaria es un criterio central de evaluación para los usuarios de plataformas sociales de video y un motivo de malestar muy frecuente para los mismos.

Inspirado en la necesidad de resolver estos problemas entendiendo que existe una oportunidad de mercado, nace la idea de *Booster* como una plataforma de videos alternativa cuyo algoritmo particular y enfoque a la experiencia de usuario permita minimizar los problemas presentados. En definitiva, *Booster* no se plantea, como una réplica de las plataformas existentes, sino como una respuesta tecnológica a necesidades no resueltas del mercado, con el fin de ofrecer un entorno, más equilibrado y más confiable tanto para quienes consumen contenido como para quienes lo producen

1.2. Objetivos

El objetivo de *Booster* es diseñar y desarrollar una plataforma de video web sustentada en un algoritmo de recomendación impulsado por la participación activa de los usuarios en la plataforma, nuevos métodos de moderación, una disminución de la cantidad de anuncios y la posibilidad de ver anuncios voluntarios, una experiencia de usuario basada en métodos de gamificación y la habilidad de crear y encontrar comunidades de usuarios y contenido más fácilmente.

De esta forma, *Booster* está orientado a responder a una necesidad creciente del servicios de video bajo demanda, con el objetivo de ofrecer a creadores y usuarios un entorno más transparente, y más eficiente que las plataformas dominantes actuales.

Esta necesidad surge de la acumulación de problemas que afectan tanto la experiencia de consumo como la de la creación de contenidos, entre ellos las dificultades de monetización para los creadores, la opacidad algorítmica en la distribución de contenidos, la formación de cámaras de eco, la reproducción de sesgos en los sistemas de recomendación, la abundancia de contenido generado por inteligencia artificial sin mecanismos suficientes de identificación y la percepción de censura injustificada derivada de procesos de moderación poco claros. En este contexto, *Booster* se propone como una solución tecnológica capaz de abordar dichos problemas mediante un algoritmo que optimice la personalización y el descubrimiento de contenido.

En consecuencia, *Booster* no se plantea únicamente como una nueva plataforma de distribución audiovisual, sino como una propuesta de arquitectura digital orientada a resolver necesidades reales del mercado.

Para desarrollar esta plataforma, se implementará una página web ya que de esta forma se puede llegar a un público más amplio y se puede implementar tecnologías modernas que faciliten el desarrollo. Además, se implementará un sistema de procesamiento y transcodificación de videos que permitirá a los usuarios subir sus propios videos y que estos puedan ser reproducidos en la plataforma.

1.3. Estructura de la memoria

El trabajo fin de grado se organiza en una secuencia de capítulos que conduce al lector desde la comprensión del problema hasta la formulación, diseño y

validación de la solución propuesta.

- **Capítulo 1 - Introducción:** Este capítulo presenta el contexto del trabajo, el estado actual de las plataformas de video bajo demanda, los problemas que enfrentan los usuarios y creadores, y los objetivos que se persiguen con el desarrollo de *Booster*.
- **Capítulo 2 - Estado del Arte:** En este capítulo se realiza un análisis detallado de las plataformas de video bajo demanda existentes, prestando atención al dominante YouTube y a otras alternativas como Rumble, Odysee y Peertube. Se pretende identificar las fortalezas y debilidades de cada una, las tecnologías que emplean, y las oportunidades de mejora.
- **Capítulo 3 - Tecnologías Estudiadas:** En este capítulo se detalla el análisis tecnológico que fundamenta al proyecto *Booster*. Se describen las principales tecnologías web seleccionadas y las descartadas para el objetivo del proyecto, un análisis de proveedores de servicios, elecciones de terceros. La finalidad es mostrar que las decisiones técnicas adoptadas responden a criterios de escalabilidad y usabilidad.
- **Capítulo 4 - Requisitos del Sistema:** En este capítulo se indican los requisitos funcionales y no funcionales de *Booster*. Se especifican las capacidades mínimas que ofrece la plataforma, las condiciones de rendimiento esperadas, las restricciones del sistema y las necesidades de usuarios y creadores.
- **Capítulo 5 - Innovaciones:** Se describen las innovaciones tecnológicas, de diseño y de experiencia de usuario que se han implementado para solventar los problemas identificados. Se detallan los aspectos novedosos de la plataforma y su factor diferencial frente a las alternativas existentes.
- **Capítulo 6 - Diseño y Arquitectura:** En este capítulo se presenta la arquitectura general de *Booster* y el de procesamiento y transcodificación de videos, los componentes principales del sistema, su interacción y el diseño de la interfaz de usuario. Aquí se explican, entre otros elementos, el algoritmo de recomendación, los sistemas de interacción como comentarios y XP, la moderación de contenidos y la experiencia general de usuario, búsqueda semántica de videos.
- **Capítulo 7 - Desarrollo:** Se detalla la implementación de ciertos componentes de la aplicación y su integración, los retos técnicos enfrentados y las soluciones adoptadas.
- **Capítulo 8 - Pruebas, Validación y Riesgos de Seguridad Detectados:** Este capítulo recoge las pruebas realizadas sobre la plataforma, tanto desde el punto de vista funcional como de seguridad, desempeño y experiencia de usuario. Se detallan medidas de seguridad tomadas, ataques reales detectados y mitigados, y se presentan los resultados de las pruebas.
- **Capítulo 9 - Conclusiones:** Se sintetizan los principales resultados del trabajo, evaluando qué se ha conseguido con el desarrollo de *Booster* según

los objetivos planteados. Asimismo, se valoran las aportaciones del proyecto, sus limitaciones y las posibles mejora o ampliaciones de la plataforma.

- **Capítulo 10 - Análisis de Impacto:** Se realiza un análisis del impacto del proyecto, considerando sus posibles implicaciones tecnológicas, económicas y medioambientales.

Capítulo 2

Estado del Arte

Este capítulo constituye una recopilación de diversas tecnologías disponibles para implementar un servicio web de streaming de videos. Asimismo, se exploran distintos modelos de arquitectura empleados por las aplicaciones que ya brindan este servicio.

2.1. Procesamiento, Transcodificación y Entrega de Video

En esta sección se hace un análisis de las herramientas más comunes para procesar, almacenar, transcodificar y entregar los videos en la plataforma.

2.1.1. FFmpeg

FFmpeg es una herramienta de código abierto utilizada para el procesamiento de contenido multimedia, en particular audio y video. Es empleada en una amplia variedad de tareas y aplicaciones como:

- **Transcodificación:** FFmpeg puede convertir videos entre distintos formato, códecs ¹ y resoluciones, lo que es esencial para garantizar la compatibilidad con diferentes dispositivos y plataformas. Por ejemplo, se puede convertir un video en formato *.mov* a *.mp4*, o extraer el audio de un archivo *.mp4* a *.mp3*.
- **Compresión y Optimización:** Permite reducir el tamaño de archivos sin perder calidad significativa ajustando la resolución, códec y el bitrate ². Esto es esencial para mejorar la experiencia de usuario. Por ejemplo, en este tipo de servicios se suele utilizar **Adaptive Bitrate**. Esto consiste en guardar varias copias del mismo video con diferentes calidades y tamaños

¹Se entiende por códec a aquella tecnología software o hardware que permite comprimir o descomprimir archivos multimedia.

²Se entiende por bitrate (tasa de bits) como la cantidad de información que se transmite por unidad de tiempo.

y, de esta forma, el reproductor puede seleccionar la versión más adecuada según la velocidad de conexión del usuario.

- **Edición de video:** Permite realizar tareas básicas de edición como recortar la duración, unir varios videos, rotar, etc.
- **Streaming:** FFmpeg puede ser utilizado para transmitir video en tiempo real a través de protocolos como RTP o UDP.



Figura 2.1: Logo de FFmpeg

Ventajas

La principal ventaja de FFmpeg es su versatilidad y flexibilidad. Puede aplicar filtros complejos, producir salidas en distintos formatos, ajustar la resolución, bitrate, códecs, frame rate y parámetros de la codificación dentro del mismo flujo de trabajo. Eso lo vuelve muy útil para transcodificación, remuxing, extracción de audio, recortes, subtítulo, entre otros casos de uso en el procesamiento de video.

Es una herramienta *open source* lo que implica que reduce la dependencia de un tercero, está disponible gratuitamente y está bien documentada. Todo esto facilita su integración en cualquier tipo de proyecto, evitando así el *vendor lock-in*.

La herramienta destaca por ser veloz y eficiente, implementando algoritmos avanzados de procesamiento de video. Además, es compatible con una amplia variedad de formatos y códecs y tiene soporte para diversos sistemas operativos. Por estas razones, FFmpeg es una opción popular y altamente utilizada en la industria para el procesamiento de video.

Desventajas

La principal desventaja de la herramienta es su complejidad. Requiere un conocimiento técnico profundo para su uso efectivo. A esto, se le suma su sintaxis compleja y la gran cantidad de opciones disponibles, lo que puede resultar abrumador para los usuarios sin experiencia.

FFmpeg es el ganador en términos de flexibilidad y control, pero sin embargo, cuando el objetivo es la velocidad extrema o el ahorro masivo de energía, el hardware dedicado o las soluciones implementadas desde cero pueden superar a FFmpeg. Por ejemplo, el chip personalizado de Google Argos VCU (Video Coding Unit) supera a FFmpeg en velocidad, eficiencia de cómputo y ahorro energético optimizando la capacidad de los centros de datos de YouTube. [9]

Servicios que emplean FFmpeg

- Netflix, que utiliza esta herramienta para la transcodificación de sus videos.
- Youtube, que incorpora FFmpeg y tecnologías personalizadas basadas en FFmpeg para procesar contenido multimedia.
- Meta, que también emplea FFmpeg para la transcodificación en plataformas como Facebook e Instagram.
- TikTok, que también utiliza FFmpeg para procesar los videos junto con tecnologías personalizadas.
- Nasa, utiliza FFmpeg en algunos de sus rovers marcianos para comprimir los videos capturados por las cámaras de estos robots antes de ser enviados a la Tierra.

2.1.2. MPEG-DASH

MPEG-DASH [10] (Dynamic Adaptive Streaming over HTTP) es un estándar internacional de transmisión de video sobre HTTP, enfocado en una distribución eficiente definido en la familia **ISO/IEC 23009**. Para ello, implementa técnicas de **Adaptive Bitrate** las cuales permiten ajustar automáticamente la calidad de reproducción, cambiando parámetros como el bitrate y la resolución del video según las condiciones de red y las capacidades del dispositivo del usuario. De esta forma, se garantiza una experiencia de visualización fluida y sin interrupciones, incluso en conexiones lentas e inestables.

Ventajas

La principal ventaja de MPEG-DASH es su escalabilidad y eficiencia. Al hacer uso de HTTP, se puede aprovechar la infraestructura existente de servidores web y CDNs para distribuir el contenido eficazmente a una audiencia global. Además, al implementar Adaptive Bitrate el video se divide en segmentos pequeños y se reduce la latencia.

Asimismo, MPEG-DASH es ampliamente compatible con una variedad de códecs, formatos de videos y dispositivos, lo que lo hace una opción versátil para la distribución de contenido multimedia.

Desventajas

MPEG-DASH, puede ser complejo de implementar y configurar. DASH, suele exigir una configuración cuidadosa de los servidores y los reproductores de video.

Aunque DASH funciona bien para la transmisión de video, no es la mejor opción para escenarios donde se requiere una latencia sumamente baja. En tales casos, se puede optar por utilizar Low-Latency DASH (LL-DASH) u otros protocolos alternativos como WebRTC.

Otra desventaja importante es que este protocolo no tiene soporte nativo en Apple, lo que limita su incorporación en este tipo de dispositivos.

Servicios que implementan DASH

DASH es un estándar abierto y empleado en gran cantidad de servicios de streaming. Algunos de estos son:

- YouTube es uno de los casos más importantes. Google informa a los desarrolladores que YouTube utiliza MPEG-DASH en su plataforma HTML5 para implementar Adaptive Bitrate.
- Brightcove, una empresa proveedora de servicios de software en la nube especializada en el alojamiento, distribución y gestión de video online, también ha informado que utiliza MPEG-DASH para la entrega de contenido multimedia.
- Bitmovin, otro proveedor de infraestructura de streaming, también ha confirmado que emplea MPEG-DASH en su plataforma para la entrega de video,.
- Vimeo, una plataforma de alojamiento de videos similar a YouTube, también ha confirmado que utiliza MPEG-DASH e implementa Adaptive Bitrate.
- Odysee y *PeerTube*, plataformas de videos descentralizadas y basadas en tecnologías blockchain, adoptan también MPEG-DASH para la entrega de contenido multimedia.

2.1.3. HLS

De forma similar a MPEG-DASH, **HLS** [11] (HTTP Live Streaming) también es un protocolo de transmisión de video desarrollado por Apple. HLS implementa técnicas de **Adaptive Bitrate** y es utilizado para la distribución de contenido audiovisual bajo demanda y en directo fundamentalmente en dispositivos Apple, aunque es compatible con otros sistemas operativos.

Ventajas

Usualmente, este protocolo divide el video en segmentos pequeños y en formato de archivo `.m3u8` que permite entregar el contenido de forma eficiente por HTTP usando CDNs y servidores web. El reproductor de video solicita los segmentos más adecuados teniendo en cuenta la calidad de conexión del usuario mejorando la experiencia de visualización.

Además, HLS es altamente escalable. Esto lo hace ideal para la distribución de contenido a gran escala siendo una opción popular para servicios de streaming en vivo y bajo demanda con momentos de elevado tráfico.

HLS, es compatible con una amplia variedad de dispositivos especialmente con los dispositivos Apple, lo que lo convierte en una opción preferida para la distribución de contenido a usuarios de iOS y macOS.

Desventajas

Una de las desventajas más notorias de este protocolo es la falta de soporte en algunos dispositivos y navegadores, especialmente en aquellos que no son de

2.1. Procesamiento, Transcodificación y Entrega de Video

Apple. Esto puede limitar su adopción en ciertos entornos y requerir la implementación de soluciones alternativas para garantizar la compatibilidad con una audiencia más amplia. [12]

Adicionalmente, HLS puede ser inadecuado para circunstancias en las que se requiera una latencia extremadamente baja, como en llamadas de video en tiempo real. Debido a su arquitectura, para este tipo de aplicaciones, otros protocolos como WebRTC podrían ser más apropiados. [13]

Servicios que implementan HLS

La gran mayoría de actores principales en el mercado de streaming implementan este protocolo. Este es el caso de:

- Apple, siendo el creador del protocolo e implementándolo en sus dispositivos y servicios de streaming.
- AWS, en su servicio de live streaming
- Mux, un proveedor de infraestructura de streaming que ofrece soporte para HLS.
- Cloudflare, que también ofrece servicios de streaming con soporte para HLS.

Cabe destacar que, en ambos protocolos HLS y MPEG-DASH, el video no se descarga por completo de una vez, sino que el reproductor va solicitando pequeños segmentos o fragmentos al servidor conforme avanza la reproducción. En otras palabras, el video está almacenado en partes pequeñas en el servidor cada una con diferentes calidades y el reproductor le solicita la parte que mejor se adapte a las condiciones de red del usuario en cada momento. Así, se evitan transmitir grandes cantidades de datos innecesarios por la red y se puede implementar un Adaptive Bitrate eficiente.

2.1.4. Object Storage

El contenido multimedia, en particular los videos, suelen ocupar un gran espacio de almacenamiento. Además, con las técnicas de **Adaptive Bitrate** es necesario almacenar varias copias del mismo video con diferentes calidades y tamaños lo cual incrementa el uso de espacio de forma considerable. Por ello, es fundamental contar con un sistema de almacenamiento eficiente y escalable. Por esa razón, se suelen emplear sistemas de almacenamiento de objetos **Object Storage** [14] como Google Cloud Storage, Amazon S3 Buckets o Azure Blob Storage que permiten almacenar grandes cantidades de datos de forma escalable y con alta disponibilidad. Esto es esencial para garantizar un acceso rápido y confiable a los videos por parte de los usuarios.

Ventajas

La mayor ventaja es su escalabilidad. Amazon S3 o Google Cloud Storage, son servicios de almacenamiento en la nube que permiten almacenar y recuperar

Capítulo 2. Estado del Arte

cantidades masivas de datos solucionándole al usuario complicaciones con la gestión de particiones, discos, entre otros.

Estos servicios suelen ofrecer una alta disponibilidad y durabilidad, lo que prácticamente garantiza que los datos estarán siempre accesibles y protegidos contra pérdidas.

Desventajas

Aunque son capaces de almacenar grandes cantidades de datos, el acceso a los mismos no es tan rápido. Los Object Storage no están pensados para accesos con latencias extremadamente bajas. Por esa razón, suelen ser utilizados en conjunto con CDNs que permiten entregar el contenido de forma más eficiente a los usuarios finales.

Servicios de Object Storage

Todos las grandes plataformas de streaming emplean Object Storage para almacenar los videos. Los servicios más comunes para implementar esta tecnología son:

- Amazon S3 Buckets, es el ejemplo más conocido y es extensivamente usado por empresas y servicios como Netflix, Disney+, Twitch, Hulu y Prime Video para almacenar su contenido.
- Google Cloud Storage, es otro servicios de Object Storage empleado por YouTube, Spotify, Vimeo, entre otros.
- Azure Blob Storage, otro servicio de Object Storage utilizado por BBC Player, NBA, Real Madrid, entre otros.

2.1.5. CDN - Content Delivery Network

Un Content Delivery Network (CDN) [15] es una red de servidores distribuidos geográficamente que trabajan juntos para entregar contenido de manera rápida y eficiente a los usuarios finales. Un usuario, al solicitar un video, es atendido por el servidor más cercano a su ubicación, con el objetivo de reducir la latencia, aumentar la escalabilidad y mejorar el rendimiento del servicio. Su uso, se vuelve necesario en plataformas con alta demanda y con usuarios distribuidos geográficamente, donde depender exclusivamente del servidor de origen generaría mayores tiempos de respuesta, mayores costos y una sobrecarga innecesaria sobre la infraestructura principal.

De este modo, las CDN funcionan como una caché de contenido de los Object Storage, replicando o almacenando temporalmente los archivos en distintos nodos de la red para servirlos de manera más eficiente a los usuarios finales. Por tanto, los Object Storage cumplen la función de almacenamiento central del contenido, mientras que la CDN asume la función de distribución acelerada.

2.1. Procesamiento, Transcodificación y Entrega de Video

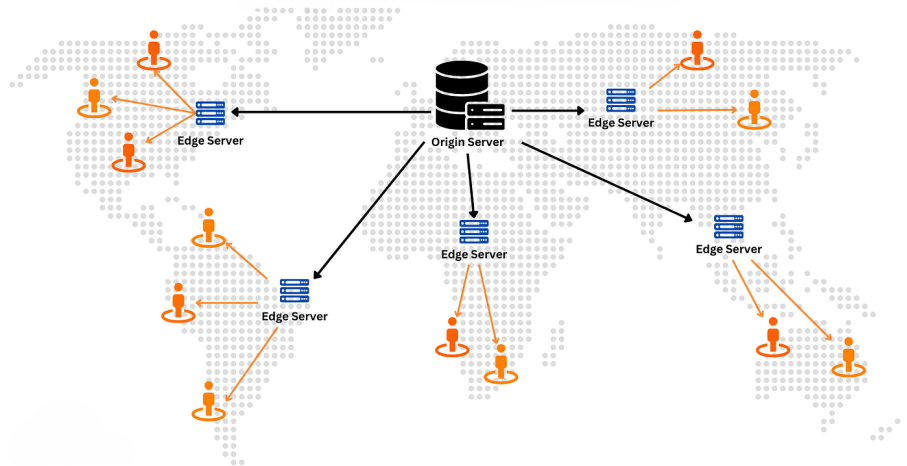


Figura 2.2: Diagrama de un CDN

Ventajas

Claramente, la principal ventaja de los CDN es que reducen la latencia y mejora el rendimiento del servicio. Como los servidores están distribuidos geográficamente, el usuario accede al servidor más cercano y esto, reduce el tiempo de carga, mejorando la experiencia de usuario.

Además, el uso de CDN reduce la carga en los servidores de origen y en los Object Storage. Al servir contenido desde los nodos de la CDN, el servidor principal recibe menos solicitudes directas, cumpliendo así una función de caché que alivia la infraestructura principal.

Los CDN también facilitan la escalabilidad del servicio. En momentos de alta demanda, distintos CDN pueden absorber el tráfico adicional permitiendo que el servicio se mantenga estable y sin depender exclusivamente del servidor de origen.

Por otra parte, un gran número de CDN suelen ofrecer capacidades adicionales de seguridad. Esto incluye protección contra ataques DDoS, firewalls (WAF) y otras medidas de seguridad que ayudan a proteger el contenido y la infraestructura del servicio.

Desventajas

La desventaja más evidente es el costo. Aunque mejora la experiencia de usuario, facilita la escalabilidad y ofrece seguridad adicional, los CDN suelen tener precios por transferencia, almacenamiento y solicitudes elevados, lo que puede resultar costoso para servicios con grandes volúmenes de tráfico y contenido.

Asimismo, los CDN introducen complejidad adicional en la arquitectura del servicio. Se deben definir políticas de caché, gestionar la invalidación de contenido y configurar adecuadamente la distribución del contenido entre los distintos nodos. Incluso, puede existir la posibilidad de servir contenido obsoleto o incorrecto

si no se gestiona adecuadamente la sincronización entre el servidor de origen y los nodos de la CDN. Por tanto, el uso de CDNs requiera una gestión constante y una planificación rigurosa añadiendo complejidad a la arquitectura.

Servicios que emplean CDNs

A pesar de sus desventajas, el uso de CDNs es muy común en la industria, especialmente en el servicio de streaming. Normalmente, cada servicio utiliza sus propios CDNs para evitar la dependencia de un proveedor y ahorrar costos. Sin embargo, algunos de los servicios de CDNs más populares son:

- Cloudflare CDN, es un servicio de CDN utilizado por empresas como HyperConnect para el streaming de video y comunicaciones en tiempo real a escala global.
- Amazon CloudFront, es otra opción popular para entregar sitios web, APIs, archivos y streaming de videos globalmente.

2.1.6. Colas de Procesamiento

El procesamiento y la transcodificación de video son tareas que suelen tomar un largo tiempo en completarse. Por esa razón, cuando un usuario sube un video, en lugar de forzarlo a esperar, se utilizan colas de procesamiento. En vez de hacer las operaciones lentas en la misma petición HTTP, el sistema registra un trabajo pendiente en una cola y este espera a ser atendido por un worker. A continuación, se realiza este trabajo y se reportan los resultados al finalizar. Así, se consiguen desacoplar procesos lentos de otros más rápidos que de lo contrario, causarían una mala experiencia de usuario.

En el contexto de procesamiento de video el esquema sería el siguiente:

- 1. El usuario sube el video
- 2. El sistema guarda el archivo en un Object Store
- 3. Se crea un job en la cola de procesamiento
- 4. Un worker toma el trabajo pendiente
- 5. Se ejecuta la transcodificación de video, se generan copias del video, se comprime, etc.
- 6. Al terminar se notifica al sistema

Existen diversas formas de implementar una cola de procesamiento. Por ejemplo, RabbitMQ, Kafka, Amazon SQS (Simple Queue Service) suelen ser opciones comunes.

Ventajas

La ventaja principal es que mejoran la experiencia de usuario. La subida de video suele ser un proceso rápido mientras que la transcodificación puede tardar minutos e incluso horas en finalizar.

Además, las colas de procesamiento facilitan la escalabilidad. Este modelo permite responder de forma efectiva ante picos de alta demanda sin colapsar el *backend*: los trabajos permanecen en un estado de espera en la cola. Para evitar que haya saturaciones o una gran cantidad de trabajos pendientes en ser atendidos, una solución común es añadir más workers según sea necesario.

Por otra parte, también favorecen la priorización de trabajos, por ejemplo, dando paso a videos Premium o urgentes antes que el resto.

Desventajas

La desventaja más clara es que las colas de prioridad añaden complejidad a la arquitectura. Esto quiere decir que además de operar con las APIs y las bases de datos, ahora es necesario gestionar workers, reintentos, resolver duplicados, timeouts, entre otros.

Servicios de Colas de Procesamiento

Cada servicio de streaming suele implementar su propia solución de colas de procesamiento adaptada a sus necesidades. Sin embargo, existen soluciones comunes como:

- Amazon SQS, es una cola de procesamiento general ofrecida por Amazon y utilizada por servicios como Fox Sports u otros clientes.
- RabbitMQ, es otra opción popular utilizada por empresas de broadcasting de contenido como Norwegian Broadcasting Corporation (NRK).
- Apache Kafka, es una plataforma de streaming de eventos ampliamente utilizada por empresas como Netflix, LinkedIn, entre otros.

2.2. Análisis de Proveedores

En esta sección se hace un estudio de los principales proveedores de infraestructura de streaming que se han considerado e implementado en el proyecto. Se analizan sus características, ventajas, desventajas, costos, facilidad de implementación e integración, entre otros aspectos relevantes.

2.2.1. Amazon Web Services y solución propia

Esta alternativa consiste en implementar una solución propia utilizando los servicios de Amazon Web Services (AWS) para cada una de las etapas del proceso. Esto implica, implementar desde cero la lógica para el almacenamiento, entrega de video, transcodificación y procesamiento, generación de miniaturas y carga de videos al servidor.

En este proyecto, se ha implementado una solución propia utilizando AWS la cual será descrita más en detalle en el capítulo de arquitectura en donde se utilizan todas las tecnologías expuestas en la sección anterior. Esta solución, aunque es más compleja, permite un control total sobre el funcionamiento del servicio.

poner capítulo arquitectura y donde Explico Docker?

Además, es útil entender cómo los servicios proveedores de infraestructura de streaming como Mux o Bunny.net, implementan su solución.



Figura 2.3: Logo de AWS

Ventajas

La principal ventaja de utilizar un pipeline propio es que se tiene un control total sobre el funcionamiento del servicio. Esto permite ajustar cada etapa a las necesidades específicas del proyecto. Por ejemplo, se pueden ajustar los servicios de AWS para que se ajusten a la escala del proyecto, realizar el procesamiento de video de forma personalizada, entre otros.

A su vez, se evita la dependencia de un proveedor de infraestructura de streaming separado. Con esto se evita el *vendor lock-in* y se tiene la libertad de cambiar o ajustar los servicios de AWS según sea necesario sin depender de las limitaciones de un proveedor externo.

Asimismo, AWS es un servicio usado extensivamente en la industria, con una disponibilidad y escalabilidad probada.

Desventajas

Evidentemente, la principal desventaja es la complejidad que añade. Implementar esta solución implica gestionar y configurar cada uno de los servicios de AWS. Además, requiere saber sobre cada una de las etapas del proceso en detalle y su integración. Un proveedor externo suele resolver todos los problemas de mantenimiento, almacenamiento, colas, procesamiento, mantenimiento y seguridad, mientras que con esta solución, el equipo de desarrollo es el responsable de gestionar cada uno de estos aspectos. En consecuencia, los recursos y el tiempo de implementación son mayores.

Uno tendería a pensar que esta solución es más eficiente en términos de costos, pero no siempre es así, como se demostrará más adelante.

Servicios que emplean AWS

Algunos Servicios de streaming que emplean AWS o utilizan servicios de AWS para su infraestructura son:

- Netflix, para su infraestructura de streaming, utiliza AWS para almacenar y entregar su contenido multimedia.
- Amazon Prime Video, que es el servicio de streaming de Amazon, también utiliza AWS para su infraestructura de streaming.

- Twitch, una plataforma de streaming de videojuegos, también utiliza AWS para su infraestructura de streaming.
- Vision+, un servicio de streaming de Indonesia, usa servicios de AWS.

Análisis de Costos

Para calcular el costo aproximado de esta solución, se deben considerar cada uno de los componentes del proceso por separado. Aunque más tarde, se detallará el funcionamiento y la razón de uso de cada uno de estos servicios, a continuación se hace un desglose de los costos aproximados para cada uno de los servicios:

- Amazon S3 Buckets: Suponiendo 500 GB de datos almacenados cada mes y tomando el precio para la región de España de un S3 Bucket Standard (\$ 0,023 por GB) el costo total es: $500 \text{ GB} \cdot 0,023\$ \text{ por GB} = \$11,5$ al mes.
- Amazon EC2: Suponiendo un *t3.medium* que trabaja 24 horas por día, y tomando el precio por hora de \$ 0,0416 el costo total es: $0,0416\$ \text{ por hora} \cdot 24 \text{ horas por día} \cdot 30 \text{ días por mes} = \$29,95$ al mes.
- Amazon SQS: Suponiendo 2 millones de solicitudes al mes, y tomando \$ 0,40 como el precio estándar por cada millón de solicitudes, el costo total es: $2 \text{ millones de solicitudes} \cdot 0,40\$ \text{ por millón de solicitudes} = \$0,80$ al mes.
- Amazon CloudFront: Si entrego 2TB de datos al mes y tomando el precio para la región de Europa de \$ 0,09 por GB, el costo total es: $2 \text{ TB} \cdot 1024 \text{ GB por TB} \cdot 0,09\$ \text{ por GB} = \$184,32$ al mes.

Sumando cada uno de estos costos, el costo total aproximado para esta solución es: $\$11,5 + \$29,95 + \$0,80 + \$184,32 = \$226,57$ al mes.

2.2.2. Mux.com

Mux es un proveedor de infraestructura de streaming de video orientada a desarrolladores. Permite subir videos, procesarlos, entregarlos, almacenarlos, gestionarlos, acceder a estadísticas, entre otras funcionalidades. Mux, facilita el proceso de desarrollo ya que se encarga de gestionar y mantener la infraestructura de streaming. Por esta razón, es una opción común para aquellos equipos que no dispongan de los recursos o el tiempo necesario para implementar una solución propia.



Figura 2.4: Logo de Mux

Ventajas

Capítulo 2. Estado del Arte

Reduce el tiempo de desarrollo, los recursos necesarios y la complejidad de la implementación. Mux, se encarga de resolver los problemas de almacenamiento, entrega y procesamiento de video, lo que permite a los desarrolladores centrarse en otras áreas del proyecto.

Mux, ofrece una documentación clara y una API flexible y fácil de usar con una gran variedad de funcionalidades. Tiene soporte para una amplia variedad de tecnologías web, lenguajes de programación y frameworks, facilitando su integración en cualquier tipo de proyecto. Esto lo hace ideal para equipos pequeños o para demostrar un producto mínimo viable (MVP) de forma rápida y barata.

A su vez, Mux ofrece características adicionales como un contador de visitas de los videos, análisis del tiempo de retención, geolocalización de las visualizaciones, entre otras estadísticas de video.

Desventajas

Al depender de un proveedor externo, se pierde el control sobre la arquitectura de procesamiento de los videos o posibles optimizaciones personalizadas que se deseen implementar.

Puede resultar una opción costosa a largo plazo, en especial para proyectos de gran escala o aquellos que esperen un crecimiento importante en el tráfico, volumen de datos, o videos subidos. Además, al ser un servicio de terceros, se está sujeto a las políticas de precios o cambios en la plataforma.

Servicios que emplean Mux

Mux resulta un proveedor de infraestructura de streaming popular en el sector, en particular para proyectos pequeños, startups o empresas medianas. Algunos de los servicios que emplean Mux son:

- Patreon, una plataforma de membresías, lo utiliza para integrar videos nativos en su plataforma
- Typeform, una plataforma diseñada para crear encuestas, emplea este servicio para incluir preguntas en video u otras funciones de videos en sus cuestionarios.
- Shopify, una empresa de *e-commerce*, utiliza en ciertas circunstancias la infraestructura de Mux para mostrar videos sobre los productos que vende.

Análisis de Costos

Según la página oficial de precios de Mux [16] y tomando como supuestos las cantidades de almacenamiento y transferencia anteriores:

- El tipo de recurso a transmitir es *On Demand Video*
- La resolución máxima es 1080p
- La calidad de video es *Basic*
- La cantidad de GB almacenados por mes son 500 GB .
- La cantidad de datos transferidos por mes son 2 TB.

- La cantidad de GB procesados por mes son 500 GB.

Con estas estimaciones, el costo aproximado por mes es \$20,00. Sin embargo, cabe destacar que si se considera usar una calidad de video superior como *Plus*, el precio se vería disparado hasta \$321,64 por mes.

2.2.3. Bunny.net

Bunny.net es una plataforma que ofrece varios servicios. Entre ellos, se encuentra la entrega de contenido mediante CDNs, almacenamiento de objetos, streaming de videos, optimización de imágenes, seguridad contra ataques DDoS. El principal punto fuerte de Bunny.net es su precio competitivo en comparación con otros proveedores de infraestructura de streaming. Sin embargo, su integración requiere un mayor esfuerzo de desarrollo.



Figura 2.5: Logo de Bunny.net

Ventajas

La principal ventaja de Bunny.net son sus precios reducidos. El almacenamiento, procesamiento y distribución de contenido multimedia es costoso, pero sorprendentemente, Bunny.net ofrece una solución barata en comparación con una solución propia u otros proveedores de infraestructura de streaming. Esto lo hace ideal para proyectos con una escala pequeña o para entregar un producto mínimo viable (MVP) de forma rápida y barata.

BunnyStream, el servicio de streaming de Bunny.net, integra CDN, transcodificación, procesamiento de video, almacenamiento, estadísticas avanzadas, subtítulos automáticos, personalización del reproductor, entre otras funcionalidades. Esto lo convierte en una opción interesante para startups que busquen una solución completa y económica para su servicio de streaming.

Desventajas

De forma similar a Mux, al depender de un proveedor externo, se pierde control sobre la arquitectura de procesamiento de los videos, incluyendo optimizaciones personalizadas. Para una escala más grande, puede resultar ineficiente y en este caso, sería más recomendable implementar una solución propia con servicios de AWS o similares.

Asimismo, en comparación con Mux, Bunny.net tiene una documentación más limitada y no tan exhaustiva como en el caso de Mux. Esto dificulta la integración y el uso de algunas de sus funcionalidades. Emplear Bunny.net, requiere más tiempo, recursos y esfuerzos de desarrollo que Mux, lo que puede no ser ideal para equipos pequeños o con recursos limitados.

Capítulo 2. Estado del Arte

Otro aspecto importante a destacar es que Bunny.net ofrece menos personalización que Mux. BunnyStream incorpora un reproductor de video con diversas opciones pero no siempre se puede adaptar el estilo del reproductor de forma completa y sencilla a las necesidades de una empresa.

Servicios que emplean Bunny.net

Bunny.net es una alternativa menos conocida que otras, siendo una opción más común para proyectos pequeños o medianos. Algunos de los servicios que emplean Bunny.net son:

- Yximity, una aplicación de desarrollo personal, integra BunnyStream para streaming de sus cursos en vivo y bajo demanda.
- STIRR, un servicio americano de streaming de TV y películas, emplea Bunny.net para la entrega de su contenido multimedia.
- Circle.so, una plataforma de comunidades online, utiliza Bunny.net para la entrega de videos.

Análisis de Costos

Tomando los mismos supuestos que en el caso de Mux o AWS y los datos publicados en la web oficial de Bunny.net [17], el desglose de los costos aproximados es el siguiente.

- Almacenando 500 GB por mes, con un precio de \$ 0,01 por GB, el costo total es: $500 \text{ GB} \cdot 0,01\$ \text{ por GB} = \$5,00$ al mes.
- Para un procesamiento básico de video ³ Bunny.net no cobra. Por tanto para el supuesto de 500GB procesados por mes el costo es \$ 0,00 al mes.
- Los costos de transferencia varían según la cantidad de CDNs utilizados y cuáles. Utilizando 3 CDNs se obtiene 0,005 \$ por GB. Por tanto, el precio por mes para 2 TB de entrega de datos es: $2 \text{ TB} \cdot 1024 \text{ GB por TB} \cdot 0,005\$ \text{ por GB} = \$10,24$

El costo total aproximado para esta alternativa es: $\$5,00 + \$0,00 + \$10,24 = \$15,24$ al mes, lo que la convierte en una opción increíblemente económica en comparación con otras alternativas. Asimismo, si elegimos una calidad de video superior, el precio no se vería tan afectado como en el caso de Mux.

³Un procesamiento básico incluye funcionalidades como transcodificación, Adaptive Bitrate, salidas con códecs x264 y resoluciones configurables desde 240p hasta 2160p. El plan básico no tiene códecs más avanzados ni prioridades en las colas de procesamiento.

2.2.4. Tabla comparativa de proveedores

Criterio	Amazon Web Services	Mux.com	Bunny.net
Facilidad de implementación	Baja	Alta	Media
Flexibilidad e integración con distintas tecnologías	Alta	Alta	Media
Escalabilidad	Alta	Media	Media
Costos	Medios	Altos	Bajos
Soporte	Alto	Alto	Alto
Documentación	Alta	Alta	Media
Personalización	Alta	Media	Baja
Popularidad	Alta	Alta	Baja
Funcionalidades y características adicionales	No incluye características directas adicionales para streaming, por lo que su implementación debe realizarse desde cero.	Ofrece una amplia variedad de funcionalidades, como reproductor de video, subtítulos, estadísticas y moderación de contenido.	Ofrece diversas funcionalidades, incluyendo subtítulos, reproductor de video con opciones avanzadas y estadísticas.
Principal ventaja	Personalización extrema y control absoluto sobre la arquitectura.	Resuelve la complejidad del desarrollo.	Resuelve la complejidad del desarrollo con precios bajos.
Principal desventaja	Mayor complejidad, ya que requiere más recursos, tiempo de desarrollo, un equipo de DevOps y mantenimiento continuo.	Costos elevados y pérdida de control sobre aspectos de la arquitectura.	Pérdida de control en la arquitectura y dependencia de un tercero.

Cuadro 2.1: Tabla comparativa de los proveedores y servicios considerados durante el desarrollo del proyecto

2.3. Modelos de Arquitectura

En esta sección se describen y analizan los tres principales modelos de arquitectura que se pueden emplear en un servicio de streaming de video. Estos incluyen el modelo centralizado y el modelo federado y el modelo peer-to-peer (P2P). Se explican las características de cada uno, sus ventajas, desventajas o debilidades, tecnologías típicas asociadas a cada modelo y ejemplos de servicios que integran alguno de estos en su arquitectura.

2.3.1. Modelo Centralizado

La arquitectura centralizada es el modelo más común y tradicional para plataformas de streaming de video, redes sociales y otros servicios web. En este modelo, una sola organización controla y gestiona los componentes fundamentales del sistema. Esto incluye la autenticación de usuarios, las bases de datos, la transcodificación de videos, el almacenamiento, la entrega de contenido, la moderación de contenido y los algoritmos de recomendación.

En otras palabras, esto quiere decir que la plataforma centralizada recibe los videos en servidores propios o contratados, los procesa con su infraestructura y finalmente, los entrega a la audiencia a través de su propia red de distribución.

Desde un punto de vista más técnico, la arquitectura centralizada se caracteriza por tener un *backend* monolítico o basado en microservicios, donde cada componente del sistema está gestionado por la misma organización. Además, no siempre es un único servidor central, sino que puede haber copias del mismo distribuidas geográficamente, pero el control y la gestión de los servicios está siempre bajo la misma entidad.

Normalmente, este tipo de plataformas se compone de una cadena de servicios que trabajan juntos para controlar de extremo a extremo el almacenamiento, procesamiento y la entrega del contenido.

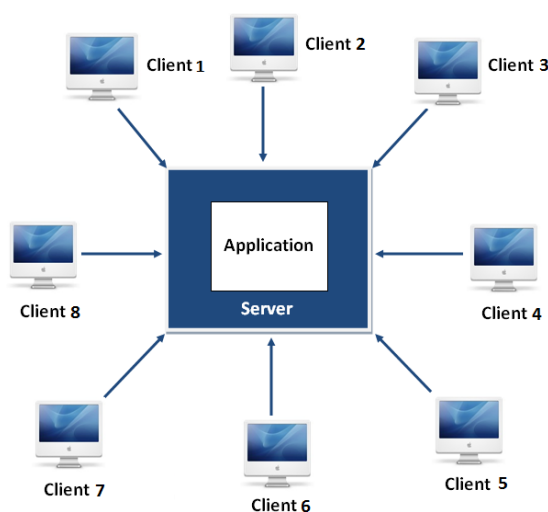


Figura 2.6: Diagrama de un modelo centralizado

Flujo de operación del procesamiento de video

El flujo comienza cuando un usuario sube un video a la plataforma. Se carga un archivo desde el *frontend* de la aplicación web o móvil hacia un servidor *backend*. A continuación, el servidor registra los metadatos del video (identificador, título, descripción, tags, visibilidad, duración, etc.) en una base de datos e intenta almacenar el video en un sistema de Object Storage.

Posteriormente, el archivo subido pasa a una cola de procesamiento, donde es-

pera a ser atendido por un worker para ejecutar tareas de transcodificación. Herramientas como FFmpeg juegan un papel fundamental en esta etapa permitiendo convertir el video a diferentes formatos, resoluciones y bitrates y segmentarlo para adaptarlo a HLS o MPEG-DASH. En paralelo, se suelen generar miniaturas, extraer metadatos adicionales, generar subtítulos, entre otras tareas.

Una vez, el video fue procesado, se distribuyen los segmentos a través de un CDN. De este modo, el video se entrega de forma eficiente a los usuarios finales.

Con los metadatos del video recién creado, un algoritmo de recomendación se encarga de sugerir el video a usuarios que podrían estar interesados en él. No obstante, en una plataforma de videos convencional, el sistema de moderación se asegura de revisar que el contenido cumpla con las políticas de la plataforma antes de ser mostrado a usuarios finales.

Ventajas

La principal ventaja es su alto nivel de control. La organización responsable es la encargada de definir cómo se almacenan los datos, qué políticas de moderación se utilizan, cómo funciona el algoritmo de recomendación, cómo se optimiza el rendimiento, políticas de seguridad, monetización.

Asimismo, comparado con otros modelos, la arquitectura centralizada es más fácil de diseñar, implementar, escalar y mantener.

Desventajas

Uno de los puntos débiles de este modelo es su fuerte dependencia de la infraestructura centralizada, que suele crear 'puntos únicos de fallo'. Si el servidor central o la infraestructura de almacenamiento experimenta problemas, el servicio puede verse degradado.

Por otra parte, en servicios de streaming de video, una arquitectura de este tipo puede generar altos costos de infraestructura y mantenimiento, especialmente a medida que la plataforma crece y atrae a más usuarios. Esto es debido principalmente a que el almacenamiento, procesamiento y la distribución tienen un alto consumo de recursos.

Servicios que implementan este modelo

La mayor parte de los servicios de streaming y redes sociales utilizan este tipo de modelo, aunque cada una con sus propias variaciones. Entre estos destacan: YouTube, Rumble, Vimeo, Twitch, Instagram, Netflix, entre otros. También suele ser una opción popular para startups ya que es más fácil de implementar e integrar *frontend*, *backend*, procesamiento y distribución de video.

2.3.2. Modelo Federado

La arquitectura federada no suele ser un modelo común en servicios de streaming de video, aunque es una opción realmente interesante. Aquí, múltiples servidores independientes llamados nodos o instancias, operan de forma autónoma aunque pueden comunicarse entre ellos mediante protocolos. Al contrario del modelo centralizado, en este caso no existe una única entidad que controle todo el sistema. Cada nodo es responsable de gestionar sus propios usuarios, almacenamiento, procesamiento, base de datos y políticas de moderación. Sin embargo, esto no significa que los nodos estén aislados: un usuario registrado en una instancia puede acceder y descubrir contenido que se encuentre en otra si ambas hablan el mismo protocolo de federación.

Desde un punto de vista técnico, cuando un usuario sube un video a una instancia, este se procesa y almacena en ese mismo nodo. Este, también se encarga de gestionar la autenticación, moderación de contenido y el procesamiento de video. Por ello, se podría considerar a cada nodo como una pequeña arquitectura centralizada e independiente.

Lo especial de este modelo es su protocolo de comunicación o federación. Esto permite que un nodo anuncie de su existencia a otros y que intercambie información. Un protocolo común es **ActivityPub**, diseñado para redes sociales federadas y que forma el **Fediverse** [18].

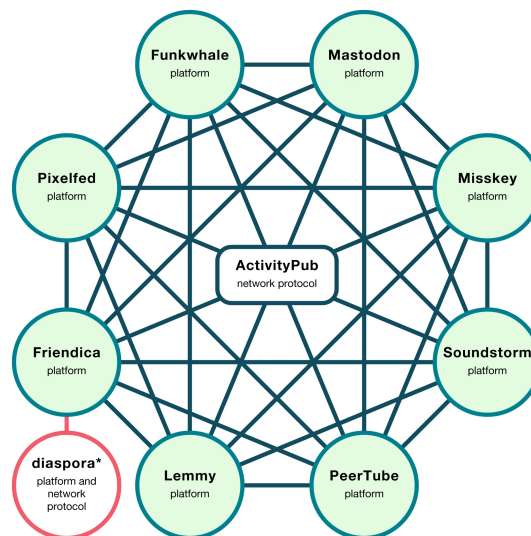


Figura 2.7: Diagrama de un modelo federado

Flujo de Operación del procesamiento de video

El flujo comienza cuando un usuario sube un video a una instancia de la red. Se carga un archivo desde el *frontend* de la aplicación hacia el *backend* de esa instancia. Esta se debió encargar de la autenticación del usuario primero y luego, el registro de los metadatos del video que se desea cargar. Además, esta misma capa se encarga de almacenar, procesar y entregar el contenido.

Una vez el video fue cargado, se sigue un flujo similar al del modelo centralizado, utilizando herramientas como FFmpeg y formatos de streaming como HLS o MPEG-DASH.

Una vez procesado el contenido, este queda disponible en la instancia donde fue subido. Gracias al protocolo de federación, este nodo anuncia sobre la existencia del nuevo video a otros nodos de la red **Fediverse**. Un protocolo usado con frecuencia es **ActivityPub**.

Cuando un usuario de otra instancia quiere acceder a ese video, se puede recuperar la referencia con el protocolo de federación y solicitar el contenido al nodo origen. Se pueden utilizar tecnologías como WebTorrent para la distribución del contenido o simplemente, crear un enlace al recurso original.

Ventajas

La ventaja más significativa es la distribución del control. No existe una única entidad que controle todo el sistema y que tenga poder absoluto sobre los datos, usuarios y contenido. Cada instancia es independiente y decide su gestión.

Otro aspecto importante a resaltar es la flexibilidad. Cada nodo puede adaptar el sistema a sus necesidades, utilizar sus propios criterios de moderación, decidir su arquitectura interna y mantener un mayor control sobre privacidad, gobernanza y gestión de datos.

Desventajas

Evidentemente, su principal desventaja es la complejidad técnica que añade. Además de gestionar el funcionamiento interno de cada instancia, se deben solucionar problemas de comunicación, sincronización y la interoperabilidad. Esto implica un esfuerzo de desarrollo adicional.

Los problemas de consistencia y disponibilidad también pueden ser un desafío. Si una instancia deja de responder, ciertos contenidos o interacciones pueden volverse inaccesibles.

Como cada instancia define sus políticas y recursos, la experiencia de usuario puede verse comprometida, a diferencia de un modelo centralizado.

Servicios que implementan este modelo

Este modelo no es común en servicios de streaming de videos. Sin embargo, existen aplicaciones que implementan este enfoque. El ejemplo más reconocido es *PeerTube*. En general, este enfoque suele ser adoptado por comunidades que priorizan descentralización, soberanía digital, independencia tecnológica y control local sobre la infraestructura.



Figura 2.8: Logo de *PeerTube*

2.3.3. Modelo Peer-to-Peer (P2P)

Esta arquitectura, aunque menos convencional, es otra alternativa para implementar una red social o un servicio streaming de videos. Este es un modelo distribuido en el que cada cliente o nodo de la red puede actuar como consumidor o proveedor de contenido. Es distinto a la arquitectura centralizada en el modo de operación. La entrega de contenido se distribuye entre varias instancias independientes y no hay una dependencia con una organización que controle toda la infraestructura principal. Por otra parte, se diferencia del modelo federado debido a los protagonistas que forman parte de la red. En el federado, la red se compone de servidores independientes y autónomas que se comunican entre sí, mientras que en el P2P, son los propios pares de la red los que toman un papel más importante.

Esto quiere decir que en P2P, los usuarios comparten fragmentos de video o archivos directamente entre ellos. Esto contrasta con el modelo centralizado donde se recibe el contenido desde la infraestructura principal controlada por la organización y además, con el federado, donde el contenido depende del nodo que lo almacena.

Normalmente, los archivos de videos en este tipo de arquitecturas se dividen en bloques o fragmentos que pueden descargarse desde varios nodos a la vez. Es decir, partes de un mismo video pueden estar distribuidas en distintos nodos de la red.

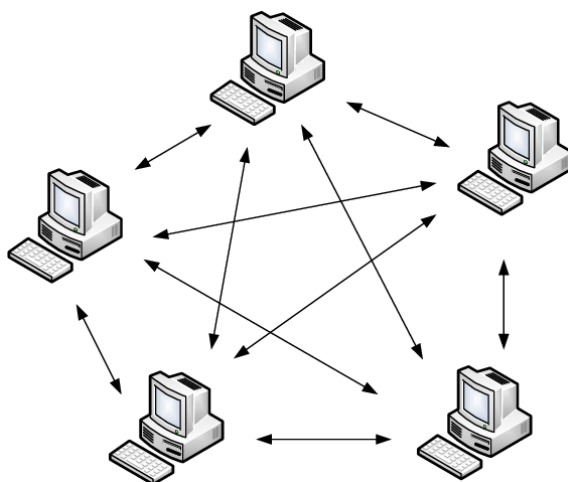


Figura 2.9: Diagrama de una red Peer-To-Peer

Flujo de procesamiento de video

El flujo comienza cuando un usuario publica un video dentro de la red. Dependiendo de las implementaciones, este archivo puede almacenarse en uno o varios nodos a la vez y dividirse en fragmentos entre ellos para facilitar su distribución. A continuación, la red utiliza mecanismos de descubrimiento para que otros pares puedan localizar el contenido y los nodos que lo tienen.

Cuando un usuario intenta acceder al video, no siempre lo descarga desde una

única fuente, ya que este puede estar fragmentado en varios nodos al mismo tiempo. Por tanto, el cliente se encarga de reconstruir el contenido a medida que recibe la información. Esto claramente se diferencia de los dos métodos anteriores en donde se tiene un origen claramente identificable.

Las tecnologías más populares para este modelos son BitTorrent, WebTorrent y IPFS. Estas, intentan distribuir los datos entre los pares de la red (clientes) mientras que el federado lo hace entre instancias (servidores independientes y autónomos) de la red.

Ventajas

Se reduce la dependencia de una infraestructura central para distribuir el contenido, lo cual puede significar una disminución en costos e incrementar la tolerancia a fallos.

Además, este modelo tiene una escalabilidad elevada. Cuantos más clientes accedan a un mismo contenido, más pares lo tendrán y lo distribuirán. A diferencia del modelo centralizado donde un aumento de la demanda incrementa la carga de un servidor central, en esta arquitectura esta carga se distribuye entre los nodos de la red.

Desventajas

La disponibilidad del contenido depende de la cantidad de pares conectados. Si pocos usuarios comparten un archivo, el acceso se volverá lento e inestable arruinando la experiencia de usuario.

Asimismo, la moderación y eliminación de contenido resulta más difícil que en los otros modelos. En los otros casos, la moderación dependía de las políticas establecidas por un servidor mientras que en este caso no.

Implementar una solución P2P añade más complejidad y esfuerzo de desarrollo que un modelo centralizado. Además del procesamiento, se deben sincronizar datos entre pares, distribuir fragmentos de archivos, recopilar información y localización del contenido distribuido por varias partes de la red, entre otros problemas de sincronización y comunicación.

Servicios que utilizan P2P

No existen servicios de streaming de videos que utilicen P2P puro. Sin embargo, una plataforma ampliamente reconocida que usa una base P2P es *Odysee*, una alternativa a YouTube descentralizada.



Figura 2.10: Logo de *Odysee*

Capítulo 3

Tecnologías Empleadas

Debido a la facilidad de implementación y a sus ventajas, se ha empleado una arquitectura centralizada para el desarrollo del proyecto según se describe en 2.3.1. No obstante, sería interesante explorar soluciones distribuidas como posibles ampliaciones.

En cuanto al procesamiento, transcodificación y entrega de video se probaron las tres alternativas mencionadas en 2.2. Para la solución propia se hizo uso de los servicios de Amazon Web Service con las tecnologías descritas en 2.1.

Para la creación de un Producto Mínimo Viable se ha decidido implementar una página web debido a su facilidad de acceso desde cualquier dispositivo y a la adecuación de los *frameworks* y tecnologías web a los objetivos del proyecto. Por ese motivo, a continuación se presentan las tecnologías utilizadas.

3.1. Backend

El backend es el conjunto de componentes que gestionan la lógica de negocio de una aplicación software, procesan datos, interactúan con la base de datos y administran la seguridad. Existe una amplia gama de tecnologías disponibles para seleccionar, especialmente en un entorno web. En esta sección, se analizarán las tecnologías preferidas y empleadas para la implementación de este servicio de streaming de videos en formato web.

3.1.1. TypeScript como lenguaje de programación

TypeScript es un lenguaje de programación desarrollado por Microsoft y es una extensión de JavaScript. Lo especial de este es que incorpora tipado estático y mejores herramientas para detectar errores durante el desarrollo y no en producción. Esto ayuda a estructurar y crear proyectos software de forma más efectiva, permitiendo un mantenimiento del código más fácil.

En el caso del proyecto Booster, TypeScript resulta una herramienta útil debido a que el sistema integra múltiples capas de lógica como gestión de usuario,

Capítulo 3. Tecnologías Empleadas

publicación y edición de metadatos de videos, autenticación, creación de comunidades y relación con la base de datos y el servidor. Por tanto, en este tipo de aplicaciones con un gran número de objetos de datos estructurados y bien definidos (usuario, videos, comunidades), el uso de tipado fuerte ayuda a reducir inconsistencias, detectar errores más fácilmente y mejorar la mantenibilidad del código.

Más técnicamente, a diferencia de JavaScript, en TypeScript se pueden definir estructuras complejas y detectar errores de forma más clara. A modo de ejemplo, se pueden declarar variables de la siguiente forma:

```
// con ':' se indica el tipo de la variable  
//a diferencia de JavaScript donde solo se incluye un let.  
saludos: string = "Hola";  
contador: number = 42;  
isFun: boolean = true;  
notas: number[] = [10,10,9.8];  
  
const user = {  
  firstName: "Samuel",  
  lastName: "Caraballo",  
  role: "Autor",  
}  
  
console.log(user.name) // Property 'name' does not exist on type  
                        // '{ firstName: string; lastName: string;  
                        //role: string; }'.
```

De esta forma, se pueden detectar errores durante el desarrollo que por el contrario, serían más complejos de identificar. Es común, olvidarse del tipo de una variable e intentar hacer operaciones no permitidas para ese tipo. TypeScript, lo detecta automáticamente pero JavaScript, no es consciente de esto y en consecuencia se generan errores difíciles de detectar.



Figura 3.1: Logo de TypeScript

Ventajas

Su principal ventaja es la detección temprana de errores. Esto es importante en proyectos con una base de código grande, donde un pequeño fallo en la estructura de datos puede provocar errores más importantes y arruinar el servicio.

Asimismo, el uso de typescript mejora la escalabilidad del proyecto permitiendo mantener un código más consistente y legible.

El uso de TypeScript es importante en la industria y por esa razón, tiene una gran integración con herramientas modernas de desarrollo de aplicaciones web como Next.js, Drizzle, tRPC, entre otros.

Desventajas

Emplear TypeScript, requiere destinar más tiempo a la definición de tipos, lo cual incrementa el tiempo de desarrollo porque requiere mantener precisión y consistencia en todo el proyecto. Por tanto, esto añade complejidad adicional en comparación con JavaScript.

Uso de Typescript

TypeScript, se ha consolidado como una de las tecnologías más utilizadas en el desarrollo web actual. Es el lenguaje por excelencia para la capa *backend* en plataformas SaaS, redes sociales y otras aplicaciones. La necesidad de compartir modelos de datos es un requerimiento frecuente en proyectos grandes y que utilizan tecnologías modernas como React.js, Next.js, Node.js, Express, entre otros.

En Booster, al ser un proyecto de código abierto, TypeScript se emplea como lenguaje principal de programación para la integración de las distintas tecnologías, garantizar precisión y consistencia, detectar errores tempranos, mejorar la mantenibilidad y mantener la coherencia entre *backend* y *frontend*.

3.1.2. Base de Datos

La base de datos es uno de los componentes más importantes en las aplicaciones software. Esto se debe a que constituye el sistema encargado de almacenar, consultar y actualizar la información del servicio. Para los objetivos de Booster, la base de datos debe permitir realizar consultas complejas de forma eficiente, integrarse con tecnologías modernas y soportar funcionalidades avanzadas como embeddings para implementar la búsqueda semántica de videos.

Por estas razones, Booster integra una base de datos **PostgreSQL** como sistema principal. Este es un gestor de base de datos relacionales ampliamente utilizado en aplicaciones de todo tipo. Su funcionamiento está basado en el modelo relacional, donde la información se ordena y almacena en tablas, filas y columnas. Las relaciones, se modelan mediante claves primarias y foráneas. Este enfoque es el más adecuado para Booster ya que la información de usuarios, videos, comunidades es claramente estructurada. Además, este gestor de base de datos ofrece una gran cantidad de funcionalidades avanzadas, integridad referencial, un gran rendimiento en producción y una buena escalabilidad.



Figura 3.2: Logo de PostgreSQL

Uso de PostgreSQL - NeonDB

En Booster, PostgreSQL se integra a través de una plataforma de base de datos moderna y serverless diseñada para la nube conocida como **NeonDB**.

NeonDB presenta una arquitectura que separa almacenamiento y cómputo para ofrecer un escalado más flexible y administrar mejor la base de datos. Además, está especialmente diseñada para soportar funcionalidades de **Inteligencia Artificial**. En el caso de Booster, se han empleado extensiones como *pgvector* para almacenar embeddings y realizar búsquedas semánticas de los títulos y descripciones de los videos. [19] Esta capacidad, permite realizar búsquedas más inteligentes en comparación a las búsquedas literales. Con esto, se puede encontrar contenido relacionado por significado y mejorar el sistema de recomendación ante una búsqueda en lenguaje natural.



Figura 3.3: Logo de NeonDB

Ventajas

Implementar una base de datos PostgreSQL tiene diversas ventajas. Entre ellas, destaca la facilidad de administración. NeonDB ofrece un entorno gráfico para realizar consultas, ver los datos, tener un diagrama de la base de datos, entre otras funcionalidades que mejoran la gestión. Asimismo, PostgreSQL posee una documentación extensa y una gran compatibilidad con distintos ORMs y frameworks *backend*.

Desventajas

Aunque es muy versátil, cuando hay un volumen alto de datos se requiere un diseño cuidadoso y un mantenimiento constante, incrementando el esfuerzo de desarrollo y la complejidad. Con un flujo de datos grande, es necesario incluir índices y destinar más tiempo a realizar optimizaciones lo que puede requerir más recursos técnicos.

SupaBase

De forma complementaria a NeonDB, se utiliza **SupaBase**, otra plataforma de *backend* que implementa una base de datos PostgreSQL. Más concretamente, en Booster se hace uso de esta para implementar funcionalidades de chat en tiempo real. SupaBase Realtime permite transmitir eventos mediante WebSockets y estar pendiente a cambios en la base de datos en tiempo real, haciéndolo ideal para implementar chats en los canales para que sus seguidores puedan interactuar entre ellos y directamente con los creadores de contenido.



Figura 3.4: Logo de SupaBase

3.1.3. ORMs

Los ORMs (Object-Relational Mappers) son herramientas que permiten la interacción con bases de datos relacionales haciendo uso de estructuras del lenguaje de programación, en el caso de este proyecto, TypeScript. En lugar de depender exclusivamente de consultas SQL manuales, se implementan funciones que actúan como capa de abstracción y además mejoran la seguridad. Por tanto, el programador puede definir relaciones y transacciones directamente desde el código de la aplicación sin trabajar con filas y tablas SQL en cada operación.

El uso de ORMs es muy común en aplicaciones modernas y existen un gran número de tecnologías a elegir. En este proyecto se han considerado dos: Drizzle y Prisma. Siendo la primera, la opción elegida para el desarrollo.

Drizzle

Drizzle ORM [20] es un ORM orientado al desarrollo con TypeScript caracterizada por ofrecer una experiencia similar a SQL y tipada.

Drizzle permite definir el esquema de la base de datos con código TypeScript mediante un archivo llamada `schema.ts`. A partir del mismo, un desarrollador es capaz de establecer relaciones entre tablas y definir los tipos de datos de las columnas para realizar consultas tipadas. Además, permite combinar consultas SQL en sus funciones haciéndolo útil para consultas complejas y con varios pasos.

La herramienta es ideal para el proyecto Booster. Esto se debe a que Drizzle permite realizar consultas avanzadas y específicas sobre bases de datos PostgreSQL.

Ventajas

Su principal ventaja, es su tipado fuerte. Junto a TypeScript, ayuda a detectar errores durante el desarrollo y mantener la coherencia y precisión entre el esquema de base de datos y el *backend*. A su vez, ayuda a prevenir inyecciones SQL mediante su capa de seguridad y mejores prácticas.

Por otra parte, según lo menciona la documentación, tiene un diseño ligero y eficiente lo cual es favorable para el rendimiento de la aplicación.

Desventajas

Drizzle es una herramienta nueva y además tiene una curva de aprendizaje elevada. Esto se debe a que requiere conocer bien el lenguaje SQL y tener experiencia con bases de datos relacionales antes de entender el funcionamiento del nivel de abstracción ofrecido por esta herramienta. Ofrece más control pero demanda un conocimiento técnico y completo.

Uso de Drizzle

En Booster se utiliza Drizzle como el ORM principal para interactuar con la base de datos. Una de las razones principales de esta elección es debido a que con un solo archivo `schema.ts` se puede definir el esquema de la base de datos. Esto facilita la creación de tablas y relaciones y permite mantener la coherencia en todo el proyecto. Además, otro punto fuerte es su compatibilidad con las tecnologías empleadas y su capa extra de seguridad incorporada.



Figura 3.5: Logo de Drizzle-ORM

Prisma

Prisma es otro ORM popular en el desarrollo de aplicaciones TypeScript. El desarrollador define un esquema propio a partir del cual se genera un cliente TypeScript que se emplea para interactuar con la base de datos. En consecuencia, se consiguen realizar consultas con un gran soporte de auto-completado y con una sintaxis orientada al programador. Prisma destaca por su facilidad para modelar la base de datos incluyendo relaciones, entidades y operaciones comunes, así como también, su experiencia de uso.

Es una buena opción cuando se necesita entregar un producto rápido y se busca legibilidad y tener una amplia variedad de herramientas adicionales. No obstante, para el proyecto se ha optado por Drizzle ya que esta tiene una relación más directa con SQL y es más ligera, mejorando así el rendimiento de la aplicación.

Esta alternativa se menciona como una herramienta importante que se podría haber incluido y se evaluó antes de comenzar el desarrollo pero, no forma parte del *backend* del proyecto final construido.

3.1.4. Comunicación Cliente Servidor - tRPC

La comunicación cliente-servidor es esencial dentro de cualquier aplicación web moderna. En la plataforma Booster, esto permite que el *frontend* se comunique con el *backend* para recuperar datos, actualizarlos y ejecutar funciones como registro de usuario e inicio de sesión. Existen diversos enfoques, siendo APIs Rest o GraphQL las más conocidas. Sin embargo, para los efectos del proyecto se ha optado por una tecnología llamada **tRPC**. Esta, está orientada al desarrollo de RPCs (Remote Procedure Calls) con TypeScript que permite construir procedimientos de comunicación entre cliente y servidor respetando los tipos y garantizando coherencia y consistencia.

tRPC es una de las tecnologías fundamentales en este proyecto facilitando el intercambio de datos entre cliente y servidor. A diferencia de un enfoque REST, con tRPC se definen procedimientos en el servidor y el cliente se encarga de consumirlos sin necesidad de especificar un esquema de endpoints o capas adicionales de generación de tipos. Además, la utilización de esta biblioteca mejora la seguridad y la coherencia de tipos de extremo a extremo. Es decir, un tipo definido en el *backend* se pueden compartir fácilmente hacia el *frontend*, convirtiéndola en una herramienta ideal para el desarrollo *full-stack*.

Al trabajar con tRPC es conveniente distinguir los siguientes conceptos. [21]

- **Procedure:** un *endpoint* de una API. Representa una acción concreta clasificadas en: consultas, mutaciones o suscripciones.
- **Query:** Un procedimiento que obtiene datos.
- **Mutation:** Un procedimiento que modifica datos (create, update, delete).
- **Suscripción:** Un procedimiento que crea una conexión persistente y escucha cambios.
- **Router:** Un conjunto de procedimientos agrupados por un nombre. Por ejemplo la acción de modificar un video (updateVideo) está dentro de un router de videos que contiene más operaciones relacionadas con esa entidad. Por lo general, se suele tener un router por cada entidad en la base de datos.
- **Contexto:** Todo aquello accesible por un procedimiento. Usado normalmente para sesiones de estado de usuarios o conexiones a bases de datos.
- **Middleware:** Una función que puede ejecutar código antes que se realice un procedimiento.
- **Validación:** Verifica que los datos de entrada a un procedimiento cumplen con ciertas normas y son correctos.

Lo particularmente interesante de tRPC es su integración con **TanStack Query** en el cliente lo cual añade mejoras de optimización, caché, revalidación, manejo de estados de carga y error, entre otras herramientas para mejorar la experiencia de uso. Esta es una de las principales razones por las que se emplea esta tecnología, además de su integración con typescript y otras tecnologías web.



Figura 3.6: Logo de tRPC (typescript Remote Procedure Calls)

prefetch con tRPC

Este es el concepto (o método) más poderoso que tiene tRPC. Gracias a él, la experiencia de usuario se ve mejorada y los tiempos de carga reducidos.

El *prefetch* consiste en cargar datos por adelantados antes de que el cliente los necesite. En la figura, se observa el flujo de esta operación implementada en el proyecto.

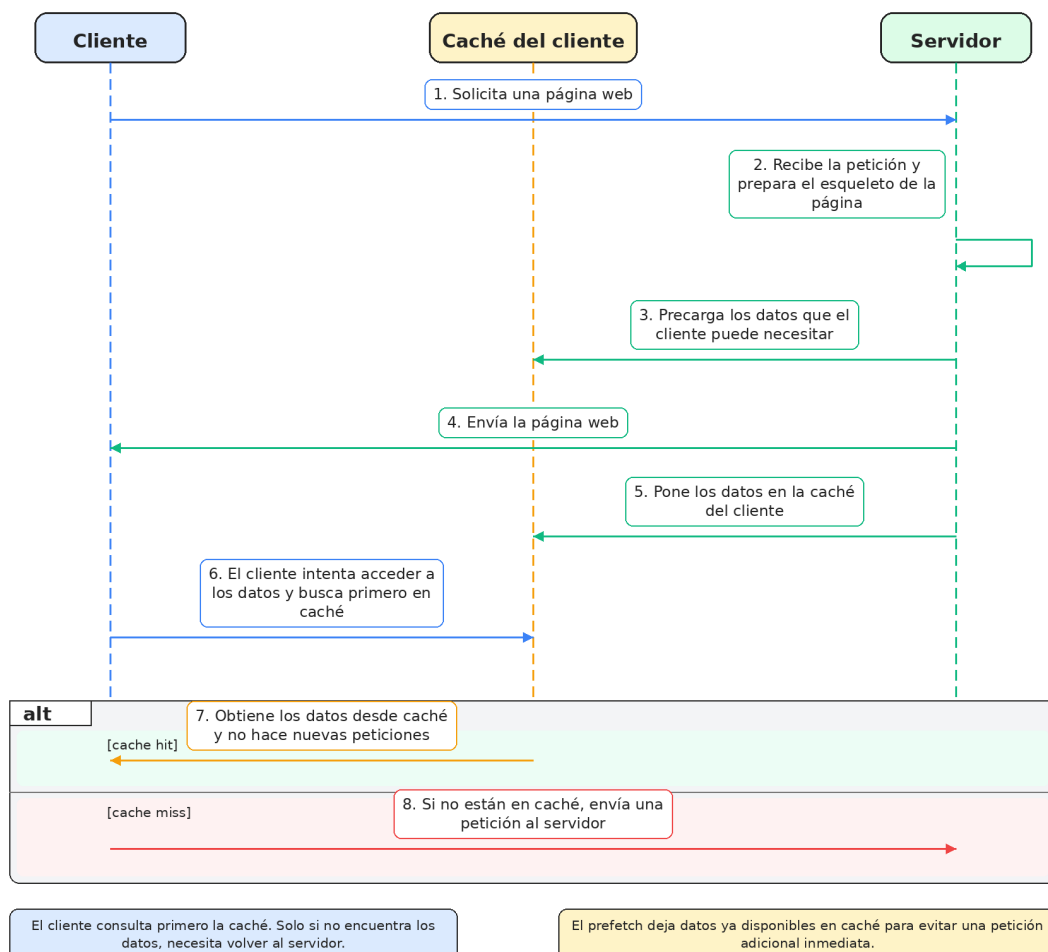


Figura 3.7: Diagrama de secuencia de un flujo de *prefetch*

Existen varias formas de aplicar el *prefetch*. En el desarrollo de esta plataforma se ejecuta una pre-carga en el servidor antes de renderizar la página para que los

datos estén disponibles cuando llegue al cliente. En Booster, esta técnica tiene un gran valor. Los usuarios están constantemente navegando entre listados de video, usuarios o comentarios por tanto, si los datos se cargan anticipadamente, el tiempo de espera disminuye creando la apariencia de una interfaz reactiva e inmediata. De esta forma tRPC tiene un doble uso: capa tipada de comunicación y un 'acelerador' de la interfaz de usuario. Por estas razones, se ha decidido utilizar tRPC para la comunicación cliente-servidor.

3.1.5. Next.js - backend

Next.js es, junto con tRPC otra de las tecnologías fundamentales de este proyecto. Este es un framework de desarrollo web basado en 3.2.1 que permite construir aplicaciones *full-stack* modernas, gestionando la interfaz de usuario y la lógica de negocio dentro de un mismo repositorio. En Booster, Next.Js se emplea para cumplir ambos de estos propósitos. Por un lado, se aprovechan sus habilidades de *frontend* pero también, se hace uso de sus capacidades de backend facilitando la experiencia de desarrollo.

Desde un punto de vista más técnico, para este proyecto se está empleando la versión moderna de Next.js que está basada en un **App Router**. Este es un sistema de enrutamiento basado en el directorio `/app`, en donde se gestionan las rutas de forma más eficiente. Es importante distinguir los siguientes conceptos:

- **Server Component:** Es un componente de 3.2.1 que se ejecuta y se renderiza **únicamente** en el servidor. Por tanto, pueden acceder directamente a recursos del *backend*.
- **Client Component:** Es un componente que se ejecuta solamente en el cliente (navegador). Normalmente, se suele reservar para interacciones con la interfaz de usuario donde la disposición de la página web puede variar.

Durante el desarrollo de Booster se combinan las habilidades de los Server Components y los Client Components para mejorar la eficiencia y optimizar el rendimiento de la aplicación. Por otra parte, gracias al *App Router* [22] se logran crear manejadores personalizados de peticiones HTTP, rutas de APIs estáticas y dinámica y crear queries de consulta. Todo esto se puede conseguir sin la necesidad de un servidor *backend* separado lo cual facilita el desarrollo y ayuda a mejorar la coherencia en la aplicación.

En Booster, esto es extremadamente útil ya que permite concentrar en un mismo entorno diversas responsabilidades. Next.Js se encarga de mostrar la aplicación, gestionar la obtención de datos, la comunicación cliente-servidor junto con tRPC y definir rutas de API y direcciones URL.

Además Next.Js contiene métodos de optimización como: incorporación de cachés, revalidación, pre-carga de enlaces, optimizaciones de imágenes, optimizaciones de código css y optimizaciones de navegación.

Ventajas

Su principal ventaja es su integración *full-stack*. Se puede construir el cliente y

Poner ref a
CSS, tail-
windcss

Capítulo 3. Tecnologías Empleadas

el servidor en un mismo repositorio facilitando el proceso de desarrollo y despliegue. Además, emplea reglas, formatos y mejores prácticas que simplifican la creación de aplicaciones web. Su integración con TypeScript y otras herramientas modernas lo convierte en una opción ideal para Booster.

Contiene una gran variedad de elementos de optimización para mejorar el rendimiento de la aplicación. En una plataforma como Booster, esto es importante ya que mejora la experiencia de usuario de forma significativa.

Otro aspecto importante a destacar es que como Next.js utiliza pre-renderizado estático (Static Site Generation SSG) ¹, los motores de búsqueda pueden indexar de forma más sencilla el contenido de la página web. Así, se consigue mejorar su descubrimiento en internet e incrementar el tráfico: Se mejora el Search Engine Optimization (SEO)

poner ref o
citar a mas
info

Desventajas

Al concentrar *frontend* y *backend* en un mismo lugar, la complejidad de los proyectos suele aumentar de forma considerable. Por el contrario, si se separa la lógica de negocio con la interfaz de usuario podría ser más fácil gestionar el proyecto por separado.

Otro aspecto importante a mencionar es su elevada curva de aprendizaje. Con los años se han añadido un gran número de operaciones y optimizaciones incrementando la cantidad de conceptos y técnicas. Esto puede significar que dominar Next.js sea una tarea difícil.



Figura 3.8: Logo de Next.js

¹Se entiende por pre-renderizado estático (SSG) a aquel proceso de generación de páginas HTML en el momento de construcción (build time). Se ejecutan comandos como `npm run build` para generar archivos estáticos (.html, .css, .js) y de esta forma, estas páginas cargan de forma instantánea al no necesitar un servidor para solicitar datos

3.1.6. Autenticación - ClerkAuth

La autenticación de usuarios es también un proceso fundamental en cualquier aplicación software. Para este proyecto se han evaluado distintas formas de gestionarlo. Entre ellas destacan la utilización de librerías como **BetterAuth** y **NextAuth**. Sin embargo, para facilitar el desarrollo, se ha optado por incluir un proveedor de autenticación y gestión de usuario llamado **Clerk** [23].

Clerk es una plataforma de autenticación y gestión de usuarios que cuenta con funcionalidades como: registro de usuario e inicio de sesión con distintas opciones, controladores de estados de sesión, perfiles de usuario, modificación de fotos de perfil, un dashboard de administración, códigos de seguridad y *multi-factor authentication*. Es una solución completa y bien preparada para poner en marcha la autenticación en una aplicación software de forma rápida y con poco esfuerzo de desarrollo.



Figura 3.9: Logo de Clerk

Ventajas

Su principal ventaja es su facilidad de incorporación en proyectos software. Permite añadir una autenticación completa y profesional en poco tiempo y es ideal para proyectos contruidos con Next.Js.

Gracias a su panel de control, la experiencia de administración es buena y fácil de realizar. Clerk permite revisar usuarios, eliminarlos, gestionar sus datos y aplicarles sanciones. Por otro lado, también cuenta con una gran variedad de opciones distintas y admite personalizar los componentes de inicio o registro de sesión que vienen por defecto, ahorrando así tiempo de desarrollo.

En definitiva, Clerk incluye una amplia gama de funcionalidades y opciones lo que reduce el tiempo de desarrollo de forma significativa y evita tener que construir desde cero los procedimientos relacionados con la gestión de usuarios, sesiones y seguridad.

Desventajas

A pesar de todas las fortalezas de Clerk, su principal debilidad es su elevado costo. Aunque ofrece un plan básico gratis de hasta 10.000 MAU (Monthly Active Users), a partir de ese momento esta opción se vuelve cara. Por esa razón, suele ser utilizado para demostrar un MVP, donde no hay un número elevado de usuarios. De lo contrario, se vuelve inviable para equipos pequeños o con recursos limitados y otras opciones como *Firebase* son más razonables.

Uso de Clerk

En Booster, Clerk se emplea para ofrecer el servicio de autenticación. Esto es debido a su buena integración con el *stack* tecnológico seleccionado y por su

dashboard de administración que facilita la gestión de usuarios y configuraciones. Sin embargo, a medida que el proyecto escale puede resultar necesario migrar a otra alternativa para ahorrar costos.

3.2. Frontend

Se define al *Frontend* como la capa software encargada de la interacción directa con los usuarios. Esto puede incluir el diseño de la interfaz de usuario, botones, menús y navegación programados con HTML, CSS y JavaScript. Estos módulos, se suelen ejecutar en el lado del cliente, para el caso del proyecto, en el navegador.

En esta sección se presentan las tecnologías seleccionadas para diseñar esta capa software recopilando herramientas modernas y compatibles con lo seleccionado en la sección anterior.

3.2.1. React.Js

React.Js es una biblioteca de JavaScript desarrollada por FaceBook en 2013 utilizada en gran medida para la construcción de interfaces de usuario (UI) mediante componentes reutilizable. Esencialmente, se basa en dividir la interfaz en pequeñas partes independientes, cada una con su lógica y representación visual y por tanto, se consiguen crear páginas por componentes separados en lugar de hacerlo todo al mismo tiempo. Además, estos elementos pueden ser reutilizados en otras partes según sea necesario. Por ejemplo, es común construir componentes para barras de navegación, botones o formularios los cuales se combinan para formar la interfaz.

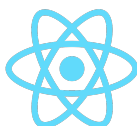


Figura 3.10: Logo de React.Js

Ventajas

La principal ventaja de React.Js es la reutilización de componentes. Esto ayuda a mantener una estructura escalable, limpia y fácil de mantener de la interfaz de usuario y el proyecto. Con esto, se logran abstraer las distintas secciones de la página web y la construcción del proyecto se vuelve más ágil y flexible.

Desventajas

React.Js no es una solución completa por sí misma. Esto quiere decir que necesita integrarse con otras herramientas para que sea útil en un proyecto.

Presenta una curva de aprendizaje significativa lo que puede resultar en un mayor tiempo de desarrollo para equipos inexpertos con esta tecnología. Además,

requiere el conocimiento y dominio de JavaScript, HTML, CSS, Node.js e incluso TypeScript.

Uso de React.Js

En Booster, se emplea React.Js para el desarrollo de las interfaces de usuario. Gracias a esta herramienta, se pueden reutilizar por toda la aplicación elementos como el reproductor de video, comentarios y barras de navegación mejorando así, la organización del código y su mantenibilidad.

3.2.2. Next.Js - frontend

Next.Js es una tecnología muy poderosa que ya se analizó en secciones anteriores (3.1.5). Aquí, se estudió su utilidad para tareas de *backend* pero también se integra bien con tareas de *frontend*.

Su funcionamiento consiste en apoyarse en componentes de React pero además, añade herramientas nuevas para el enrutamiento, generación de páginas, distintas formas de renderizado, más optimizaciones, entre otros. En cuanto al *frontend*, facilita algunas tareas y mejora la organización del proyecto ya que dispone de un sistema de rutas y páginas bien definidas con patrones claros y estructurados. Así, se consigue un código más legible, limpio y mantenible.

Booster utiliza Next.Js en el frontend para organizar los componentes y la aplicación de manera profesional y escalable. Con esto, se consigue gestionar la navegación entre vistas, mejorar el rendimiento general del sitio y su presencia en la web. Además, cuenta con una documentación extensa y clara. Otro aspecto positivo es que se integra bien con una amplia gama de tecnologías diferentes haciendo que el proyecto tenga mayor flexibilidad.

3.2.3. TailwindCSS y Shaden

Normalmente, las aplicaciones web suelen mejorar su interfaz de usuario mediante CSS (Cascading Style Sheets), un lenguaje de diseño gráfico para definir la apariencia y el estilo de los documentos HTML. Sin embargo, para este proyecto se ha optado por utilizar **TailwindCSS**, una herramienta más moderna basado en utilidades de CSS para aplicar estilos a componentes HTML.

Su funcionamiento consiste en aplicar directamente en un atributo `class` o `className` en HTML propiedades CSS (*utility classes*) predefinidas que representan características visuales específicas. Por ejemplo, el siguiente código pone a rojo el color del texto a rojo

```
...  
<p class="text-red-200">Texto rojo. </p>  
...
```

Ventajas

La principal fortaleza de Tailwind es la rapidez con la que se pueden construir interfaces. Además, las utilidades están bien definidas y con ellas se consiguen

desarrollar interfaces modernas, coherentes y visualmente atractivas.

A su vez, como los estilos se aplican directamente como atributo de los componentes HTML, esto evita tener archivos de estilos específicos para distintas partes de la aplicación. Esto mejora la organización del código en el proyecto.

Con este framework, es posible además mantener una paleta de colores consistentes con la marca a lo largo de toda la aplicación y de una forma clara y sencilla. Con estas clases además, se puede construir una aplicación visualmente responsiva para distintos tipos de dimensiones de dispositivos, mejorando así la experiencia de usuario en distintas plataformas.

Desventajas

TailwindCSS suele ser una gran opción para el desarrollo moderno de interfaces. No obstante, puede hacer que el código de los componentes HTML esté más sobrecargado. Como las clases se escriben directamente en los elementos HTML, es común tener una gran variedad de atributos distintos para un único elemento. Esto puede dificultar la lectura y la comprensión de la apariencia del componente.

Uso de TailwindCSS

Booster hace uso de las capacidades de TailwindCSS para el diseño de la plataforma. Se utiliza principalmente para mejorar y modernizar la apariencia visual de los elementos en pantalla. Asimismo, se definen colores, tamaños, tipografías y comportamientos responsivos.

Complementariamente a Tailwind, Booster incorpora **shadcn/ui** [24], un sistema de componentes R reutilizables y personalizables como botones, barras laterales, avatares, menús, diálogos, toasts, formularios, tablas, entre otros. Gracias a esto, se utilizan elementos de interfaz ya elaborados y con una apariencia muy moderna y atractiva. Esto permite, ofrecer una experiencia de usuario más profesional y mejora la **usabilidad** de la aplicación. El uso de esta librería es importante en Booster ya que se consigue una interfaz limpia, moderna y funcional sin dedicar demasiado tiempo al desarrollo de cada componente desde cero.



Figura 3.11: Logotipos de TailwindCSS y Shadcn/ui

Capítulo 4

Innovaciones

4.1. Problemas detectados y la solución de Booster

4.1.1. Problemas con el algoritmo de recomendación

4.1.2. Problemas en encontrar comunidades

4.2. Mejoras e Innovaciones

4.2.1. Mejoras en la interfaz de usuario

4.2.2. Mejoras en los métodos de moderación

4.2.3. Mejoras en los comentarios

4.2.4. Mejoras en los canales de usuario

4.2.5. Mejoras en las comunidades

4.2.6. Incorporación de gamificación

Capítulo 5

Análisis

5.1. Requisitos Funcionales

5.2. Requisitos No Funcionales

5.3. Actores

5.4. Casos de uso

5.5. Diagramas de secuencia

En esta sección se recopilan algunos de los diagramas de secuencia más importantes en la aplicación final.

5.6. Diagrama Entidad Relación

Capítulo 6

Diseño y Arquitectura

- 6.1. Patrones y mejores prácticas**
- 6.2. Diseño de la Autenticación**
- 6.3. Diseño de la Base de Datos**
- 6.4. Arquitectura de procesamiento propia**
- 6.5. Arquitectura de procesamiento con un proveedor**
- 6.6. Diseño de la interfaz de usuario**
 - 6.6.1. Comentarios**
 - 6.6.2. Interfaz de Video**
 - 6.6.3. Secciones de Video**
- 6.7. Diseño del Algoritmo de Recomendación**
- 6.8. Diseño de la búsqueda semántica de videos**

Capítulo 7

Desarrollo

Capítulo destinado a la explicación de algunos de los aspectos más importantes de la aplicación como: Implementación del procesamiento de videos propio (incluyendo código python y docker) y con proveedor (incluyendo implementación de las rutas más importantes y el TUS upload). cómo se implementan los comentarios, patrones y mejores prácticas (explicar implementación de prefetch), fetch de usuarios, fetch de videos y donación de XP, cómo se protegen rutas, cómo se protege acceso a videos, /studio.

- 7.1. Implementación del procesamiento de videos propio**
- 7.2. Implementación del procesamiento de videos con proveedor**
- 7.3. Implementación de los comentarios**
- 7.4. Patrones y mejores prácticas en código**
- 7.5. Acceder y modificar datos de usuario**
- 7.6. Acceder y modificar datos de video**
- 7.7. Medidas de seguridad**
 - 7.7.1. Protección acceso a rutas**
 - 7.7.2. Protección acceso y modificación de videos**
 - 7.7.3. Limitaciones de uso (Rate Limits)**

Capítulo 8

Pruebas, Validación y Riesgos de Seguridad Detectados

8.1. Pruebas de funcionamiento

8.2. FeedBack de Usuarios

8.3. Vulnerabilidades detectadas y medidas tomadas

Capítulo 9

Conclusiones

Resumen de resultados obtenidos en el trabajo, conclusiones sobre los mismos y potencial trabajo futuro si existiera.

Capítulo 10

Análisis de Impacto

En este capítulo se realizará un análisis del impacto potencial de los resultados obtenidos durante la realización del trabajo, en los diferentes contextos para los que se aplique:¹

- Personal
- Empresarial
- Social
- Económico
- Medioambiental
- Cultural

En dicho análisis se destacarán los beneficios esperados, así como también los posibles **efectos adversos**. Además, se harán notar aquellas decisiones tomadas a lo largo del trabajo que tienen como base la consideración del impacto.

Se recomienda analizar también el potencial impacto respecto a los Objetivos de Desarrollo Sostenible (ODS), de la Agenda 2030, que sean relevantes para el trabajo realizado (ver enlace)

¹Téngase en cuenta que no se espera que un trabajo tenga impacto en todos los contextos

Bibliografía

- [1] A. Inc. «Alphabet Announces Fourth Quarter and Fiscal Year 2025 Results», visitado 26 de mar. de 2026. dirección: https://s206.q4cdn.com/479360582/files/doc_financials/2025/q4/2025q4-alphabet-earnings-release.pdf
- [2] TikTok. «TikTok's community in Europe Tops 200 Million», visitado 26 de mar. de 2026. dirección: https://newsroom.tiktok.com/tiktok-community-in-europe-tops-200-million?from_seo_redirect=1&lang=en-150
- [3] I. Netflix. «FINAL Q4 25 Shareholder Letter», visitado 26 de mar. de 2026. dirección: https://s22.q4cdn.com/959853165/files/doc_financials/2025/q4/FINAL-Q4-25-Shareholder-Letter.pdf
- [4] I. Meta Platforms. «Meta Reports Fourth Quarter and Full Year 2025 Results», visitado 26 de mar. de 2026. dirección: <https://investor.atmeta.com/investor-news/press-release-details/2026/Meta-Reports-Fourth-Quarter-and-Full-Year-2025-Results/default.aspx>
- [5] International Energy Agency. «Energy and AI», visitado 26 de mar. de 2026. dirección: <https://www.iea.org/reports/energy-and-ai/energy-demand-from-ai>
- [6] Ofcom. «Adults' Media Use and Attitudes 2025», visitado 26 de mar. de 2026. dirección: <https://www.ofcom.org.uk/siteassets/resources/documents/research-and-data/media-literacy-research/adults/adults-media-use-and-attitudes-2025/adults-media-use-and-attitudes-report-2025.pdf?v=396240>
- [7] Ofcom. «Online Nations Report 2025», visitado 26 de mar. de 2026. dirección: <https://www.ofcom.org.uk/siteassets/resources/documents/research-and-data/online-research/online-nation/2025/online-nations-report-2025.pdf?v=409837>
- [8] X. Lu, X. Hou, N. Yuan y J. Wang. «Allow or not? Strategic choices of competing platforms in response to ad blockers», visitado 26 de mar. de 2026. dirección: <https://www.sciencedirect.com/science/article/abs/pii/S037872062500120X>

BIBLIOGRAFÍA

- [9] P. Ranganathan, D. Stodolsky, J. Calow et al. «Warehouse-Scale Video Acceleration: Co-design and Deployment in the Wild», visitado 28 de mar. de 2026. dirección: <https://gwern.net/doc/cs/hardware/2021-ranganathan.pdf>
- [10] MPEG. «MPEG-DASH», visitado 26 de mar. de 2026. dirección: <https://www.mpeg.org/standards/MPEG-DASH/>
- [11] Apple Inc. «HTTP Live Streaming», visitado 26 de mar. de 2026. dirección: <https://developer.apple.com/streaming/>
- [12] Mux Video. «Play your Videos | Mux», visitado 26 de mar. de 2026. dirección: <https://www.mux.com/docs/guides/play-your-videos>
- [13] H. P. G. T. F. Casper Haems Jeroen van der Hooft. «Hybrid Unicast–Broadcast Video Delivery for Scalable Low-Latency Live Streaming», visitado 26 de mar. de 2026. dirección: <https://dl.acm.org/doi/10.1145/3742868>
- [14] Google Cloud. «Google Cloud Storage», visitado 26 de mar. de 2026. dirección: <https://cloud.google.com/learn/what-is-object-storage>
- [15] Cloudflare. «What is a CDN?», visitado 26 de mar. de 2026. dirección: <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>
- [16] Mux Video. «Mux Video Pricing», visitado 28 de mar. de 2026. dirección: <https://www.mux.com/pricing>
- [17] Bunny.net. «BunnyStream Pricing», visitado 28 de mar. de 2026. dirección: <https://bunny.net/pricing/stream/>
- [18] W3C. «ActivityPub», visitado 28 de mar. de 2026. dirección: <https://activitypub.rocks/>
- [19] Neon. «The pgvector extension», visitado 31 de mar. de 2026. dirección: <https://neon.com/docs/extensions/pgvector>
- [20] Drizzle. «Drizzle ORM - Why Drizzle?», visitado 31 de mar. de 2026. dirección: <https://orm.drizzle.team/docs/overview>
- [21] tRPC. «Concepts | tRPC», visitado 31 de mar. de 2026. dirección: <https://trpc.io/docs/concepts>
- [22] Next.js. «Next.js Docs: App Router», visitado 31 de mar. de 2026. dirección: <https://nextjs.org/docs/app>
- [23] Clerk. «Welcome to Clerk Docs», visitado 1 de abr. de 2026. dirección: <https://clerk.com/docs>
- [24] shadcn | Vercel. «The Foundation for your Design System», visitado 1 de abr. de 2026. dirección: <https://ui.shadcn.com/>

Anexos

Apéndice A

Primer anexo

Este capítulo (anexo) es opcional, y se escribirá de acuerdo con las indicaciones del Tutor.